DevOps Course Outline

Based on the DevOps roadmap video transcript | 6 to 10 months | 6 sequential phases

Course principles

- **Foundations first.** A shaky base makes every later phase slower and more painful.
- **Concepts over tools.** Tools change, but the underlying concepts stay.
- **Projects before certifications.** Build hands-on first, then certify to prove it.
- **Combine, do not isolate.** Apply each new tool to the same growing project.
- **Depth over shallow tutorials.** Build real, end-to-end processes.
- **The 80/20 rule.** The basics of Docker and Kubernetes cover most real use cases.

Timeline at a glance

Phase	Topic	Beginner	Some experience
1	Foundations: Linux, Bash, Git	2 months	1 month
2	Cloud basics (AWS)	2 months	1 month
3	Infrastructure as Code	1 month	1 month
4	Containers and orchestration	2 months	1 month
5	CI/CD	2 months	1 month
6	Observability	1 month	1 month
Total		**10 months**	**6 months**

Phase 1: Foundations (Linux, Bash, Git)

Why: You will manage Linux-powered systems for the rest of your DevOps work. Do not skip this phase.

Modules

1. Linux fundamentals: filesystem, users and permissions, processes, package management, networking basics
2. Bash scripting: variables, conditionals, loops, functions, automating repetitive tasks
3. Git: init, commit, branching, merging, conflicts, GitHub workflows

Hands-on: Write scripts that automate a real task and keep them in a Git repository.

Milestone: Comfortable in the terminal, with a versioned scripts repo.

Phase 2: Cloud Basics (AWS)

Why: Compute, storage and networking work the same on any cloud. Learn the concepts first, then how AWS implements them. AWS is recommended for its market share and job listings.

Modules

1. Cloud concepts: compute, storage, networking
2. AWS core services: a small set is enough, so ignore the hundreds of others
3. Identity and access basics, and cost awareness

Hands-on: Deploy a simple app manually on AWS infrastructure.

Milestone: A running app on cloud infrastructure that you built by hand.

Note: Do not chase certifications here. Certify only after building real projects.

Phase 3: Infrastructure as Code

Why: Learn what IaC is, why it exists and what problems it solves, then the tool (Terraform or Pulumi).

Modules

1. IaC concepts: declarative vs manual infrastructure, repeatability, version control
2. Terraform on AWS: providers, resources, variables, state, modules
3. Automating repetitive tasks and different parts of the system

Hands-on: Recreate your Phase 2 manual setup entirely in Terraform code.

Milestone: Infrastructure that can be created and destroyed from code.

Phase 4: Containers and Orchestration

Why: One of the two most rewarding phases, where everything starts to come together. Master the basics, not every feature.

Modules

1. Container concepts: how containers work and why they matter
2. Docker: images, Dockerfiles, volumes, networking, registries
3. Kubernetes: pods, deployments, services, config and secrets

Hands-on: Containerize your app, then run it on Kubernetes.

Milestone: App running in containers, orchestrated by Kubernetes.

Phase 5: CI/CD (Automated Release and Deployment)

Why: CI/CD is the backbone of DevOps, so spend the most attention here. Understand the concepts, then pick one tool: Jenkins, GitHub Actions or GitLab CI.

Modules

1. CI/CD concepts: build, test, deploy, environments, release strategies
2. Pipeline tooling: pick one tool and go deep
3. Connecting the pipeline to containers, Kubernetes or AWS

Hands-on: Build a pipeline end to end that takes a code commit through to a containerized deployment. Reading docs or following shallow tutorials is not enough.

Milestone: Push code, and it deploys automatically.

Phase 6: Observability

Why: The final, more advanced step. Aim to understand the fundamentals and how to implement them, not to become an expert.

Modules

1. Observability concepts: metrics, monitoring, alerting
2. Prometheus and Grafana: collecting metrics and building dashboards
3. Applying monitoring to your deployed system

Hands-on: Add monitoring and dashboards to your running application.

Milestone: A fully monitored, automated system.

Running project (ties it all together)

One application evolves through every phase:

Phase	What students add
1	Scripts and a Git repository
2	Manual deployment on AWS
3	The same infrastructure rebuilt in Terraform
4	Dockerized and running on Kubernetes
5	Fully automated CI/CD pipeline
6	Prometheus and Grafana monitoring

Final deliverable: A portfolio project and a short walkthrough showing code commit to deployment to monitoring.

Personalizing the path

- **Software developers** can move faster through Git and scripting.
- **SysAdmins and network engineers** can move faster through Linux and cloud networking.
- **QA and automation engineers** can build on their scripting and pipeline experience.
- **Database administrators** can use their systems knowledge in the cloud and infrastructure phases.

Common mistakes to avoid

- Learning tools in isolation (one AWS course, one Docker course, one Terraform course) without combining them
- Collecting shallow courses and tutorials instead of building real systems
- Learning syntax without understanding the concepts behind it
- Starting a new project for every tool instead of extending one project